Rules Decide, Language Models Narrate: Fail-Closed Numeric Grounding of LLM Narration in a Flood-Response Pipeline

Nandhakumar T
Department of Information Technology
Sona College of Technology
Salem, Tamil Nadu, India
nandhakumartm93@gmail.com

Balakarthikeyan B S
Department of Artificial Intelligence and Data Science
Sona College of Technology
Salem, Tamil Nadu, India
balasan2626@gmail.com

Gowthaman J
Department of Artificial Intelligence and Data Science
Sona College of Technology
Salem, Tamil Nadu, India
gowthamanjanadhran@gmail.com

Abhishek S
Department of Artificial Intelligence and Data Science
Sona College of Technology
Salem, Tamil Nadu, India
abisheksengodan18@gmail.com

***Abstract*—Large language models (LLMs) are proposed as the voice of emergency-management software, but they can state numbers that are not in the data. We study a flood-response pipeline in which a deterministic policy chooses every action and an LLM only narrates, behind three fail-closed checks: a health-gated policy, a disagreement gate between two classifiers and a numeric guard that replaces a message containing an unsupported number by a template. We evaluate the checks on the deployed code. A composed check of the path from policy to dispatch showed that dispatch ignored the policy whenever a health check failed at high flood probability (2,807 grid cases); a one-line fix removed every such case. Of 300 replies from two open-weight models, 36.3% contained an unsupported number, 106 of 109 of them invented arrival times. The deployed guard caught 89.0% and a slot-typed guard 93.6%. A narrator that writes placeholders and leaves every number to the program, tested live, left a digit or number word in 2.1% of replies, which then fall back to the template; 1 replies that passed still did. A revised guard frozen before a third model was queried was not significantly better than the deployed guard on that model. No numeric guard sees qualitative over-claims such as "flood confirmed", which 14.3% of replies made, so our grounding claim covers numbers only; a frozen check on assertion words flags such claims. The gate held 26.9% of held-out cases and caught 40.3% of the policy model's errors. All rates are specific to our prompt, models and states.**

***Index Terms*—large language models, hallucination, fail-safe design, runtime verification, flood response**

I. Introduction

During a flood, an emergency operations centre runs on short spoken messages: the river is at this level, the forecast says this, send a team there. Software that supports the centre is increasingly asked to *speak*, and LLMs make this easy. They also produce confident statements that their input does not support [1]. On an operations net a wrong number, or a wrong claim that a flood has started, can trigger or cancel an evacuation.

Agentic systems let the model plan and call tools [2], [3]. For a safety-critical setting we take the opposite position: decisions are made by small deterministic rules that can be checked exhaustively, and the LLM only turns their state into radio traffic. This follows the view that the level of automation should be chosen per function and per risk [4] and that safety is a property of the control structure rather than of one component [5]. The design is close to a classical alerting pipeline, in which a rule fills a structured message such as a Common Alerting Protocol (CAP) alert [6]; what is new is a free-text narrator inside it. Two questions follow. Does the verified decision govern what the system does? And how often does the narrator misstate the state, and which design or check prevents it?

We answer both on an open flood decision-support prototype for the Delhi Yamuna corridor:

1) A composed check of the path from policy to dispatch, with its equivalence classes, which found that dispatch bypassed the policy and confirmed the fix.
2) A measurement of numeric grounding on 300 live replies of two open-weight LLMs and on injected errors, comparing the deployed guard, a slot-typed guard, an LLM judge and two prompt designs, including one in which the LLM writes no number at all.
3) A held-out test of a revised guard on a third model family, frozen by hash before its replies were read, with paired tests, and a frozen check for qualitative over-claims.
4) A held-out study of the disagreement gate that reports the share of errors it catches and its cost against a calibrated single model.

Algorithm 1 One pipeline tick (after the fix)

read observations; weather and dam components emit updates
$p_{\mathrm{RF}}, p_{\mathrm{XGB}}, p \leftarrow$ classifiers; $H \leftarrow$ (2)
$(h_d, h_m, h_s) \leftarrow$ health checks; $a \leftarrow \pi(p, \mathbf{h}, o)$ by (1)
if H **and not** o **then** $a \leftarrow$ REVIEW and log the hold reason
if $a \neq$ MONITOR **then** rank roads, shelters, routes
if $a \in \{\text{AUTO}, \text{OVERRIDE}\}$ **and** band(p) = auto **then** dispatch
$T \leftarrow$ LLM turns for band(p); **if not** $\Gamma(T)$ **then** $T \leftarrow$ template
broadcast state, decision and T

A companion paper on the whole decision chain reuses the dispatch result and the guard results on the development replies; this paper reports the full guard study, the prompt designs and the gate.

II. Related Work

LLM agents interleave reasoning with actions [2] and cooperate in conversations [3]; surveys describe them as memory, planner and action interface around a model [7].

Hallucination detection and guardrails. Generated text departs from its source in many ways [1]. Retrieval grounds generation in documents [8]; post-hoc checkers decompose text into atomic claims [9], sample the model several times [10], let the model verify itself [11] or ask another model to judge [12]. NeMo Guardrails programs dialogue rails and checks facts with a further model call [13], which is the setting of our LLM-judge baseline; runtime verification checks outputs against expert specifications [14], and runtime enforcement blocks unsafe agent actions [15]. Our facts are a handful of numbers in a known state, so a deterministic, typed check is possible, and so is a design in which the model writes no numbers.

Safe automation. Shields block unsafe actions of learned controllers [16], and ensembles estimate uncertainty [17]. We use disagreement between two classifiers as a hold signal.

Research gap. We found no study that measures, on an emergency system's deployed code, whether a verified decision governs actuation and how often live LLM narration misstates numeric state, comparing deterministic checks, a model judge and a numbers-free narrator.

III. Architecture

Fig. 1 shows the system. A FastAPI backend runs one pipeline per tick, in live mode (Open-Meteo weather and river data) or in simulation mode (a digital twin of the corridor). The code calls its eleven components agents (weather, dam, flood risk, traffic, shelter, evacuation, rescue, ambulance, disaster team, administrator and a monitor of sudden changes). Ten of them inform, rank, propose or narrate; only the policy (1) decides, and dispatch executes only what it allowed. The system is therefore a rule-based pipeline with an LLM narrator, and we use the code's names only to identify components. Every event goes to a MongoDB log and a WebSocket stream. Algorithm 1 gives one tick; narration runs last, after the decision is logged, so a narration failure can delay a radio message but cannot change a decision.

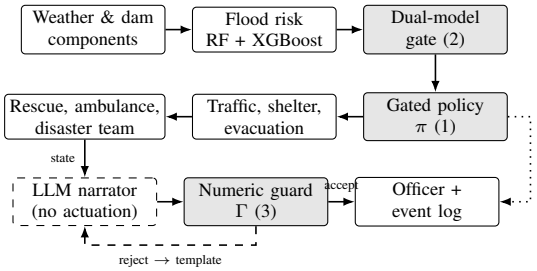


Fig. 1. Pipeline. Shaded blocks can only hold, downgrade or reject; none can trigger an action. The LLM (dashed) receives the shared state and has no actuation rights. The dotted link carries every policy decision and its reason to the officer.

A. Decision Policy

The flood-risk component receives a flood probability p and three health flags: h_d (weather and river data available), h_m (model loaded and returned a value) and h_s (database connected). An operator can set an override o, and a flag A enables automation. With $\tau_m = 0.50$ and $\tau_a = 0.60$, served from one configuration module,

$$\pi(p, \mathbf{h}, o) = \begin{cases} \text{UNAVAILABLE} & p \text{ undefined} \\ \text{OVERRIDE} & o \\ \text{MONITOR} & p < \tau_m \\ \text{AUTO} & p \geq \tau_a \wedge h_d \wedge h_m \wedge h_s \wedge A \\ \text{REVIEW} & \text{otherwise.} \end{cases} \tag{1}$$

B. Classifiers and the Disagreement Gate

Two classifiers score the same inputs: a Random Forest [18] and an XGBoost model [19], the latter supplying p. Both were trained on the INDOFLOODS catalogue (4,548 flood events at 155 gauges) to predict whether an event is *severe*; the label is the catalogue's flood type, and 35.8% of events are severe. Their 55 numeric and 4 categorical inputs are rainfall over the last 1 to 10 days with engineered ratios, the season, catchment attributes and a gauge prior. A stratified random 80/20 split of events gave a held-out ROC-AUC of 0.818 (RF) and 0.806 (XGBoost) and an accuracy at τ_a of 75.5% and 75.2%. The "flood probability" that the system displays is therefore the probability that a flood event at the corridor's gauge would be severe. Automatic dispatch is held when

$$H = \big[\, |p_{\mathrm{RF}} - p_{\mathrm{XGB}}| \geq \delta \,\big] \\ \vee \ \big[\min(p_{\mathrm{RF}}, p_{\mathrm{XGB}}) < \tau_a \leq \max(p_{\mathrm{RF}}, p_{\mathrm{XGB}}) \big], \tag{2}$$

with $\delta = 0.15$; the second, *straddle* term flags models on opposite sides of the threshold.

C. LLM Narration and the Numeric Guards

Agent conversations are generated by an LLM through an external API. The band follows p (calm below 0.40, watch below 0.50, administrator review below 0.60, automated response above) and sets who speaks and the tone; the action is still set by π. The prompt lists the current facts (probabilities, rainfall,

river level, barrage stage, time and place), states the band's rule and tells the model to use only those numbers. The rule of the automated band reads "Above 60%: flood occurred", and the template that replaces a rejected reply begins "Flood has occurred"; Section V-F returns to this wording.

Let $\mathcal{N}(s)$ be the decimal numbers in a string s and F the prompt text up to the recent-history block. With $\tilde{F} = \{f, 100f, f/100 : f \in \mathcal{N}(F)\}$, the deployed guard accepts turns T when

$$\Gamma(T) = \bigwedge_{t \in T} \bigwedge_{n \in \mathcal{N}(t)} \exists f \in \tilde{F} : \ |n - f| \le \tfrac{1}{2} 10^{-d(n)}, \quad (3)$$

where $d(n)$ is the number of decimals written in n, so honest rounding passes. The guard is untyped: a number written as a word is invisible to it, and a value drawn uniformly from an integer range $[a, b]$ passes with probability

$$P_{\text{miss}} \approx |\{\lfloor f \rceil : f \in \tilde{F}\} \cap [a, b]| \,/\, (b - a + 1). \quad (4)$$

Slot-typed guard. Γ_{T} reads facts from the structured state and binds each number to a quantity before checking it: number words become digits, clock times must equal the current time, a number with "%" must match a model probability or a threshold of the band, one with "mm" the rainfall and one with "m" a water level. An unbound number is rejected. A revised version, $\Gamma_{\mathrm{T}}^{(2)}$, adds procedural numbers (reporting intervals, deadlines given as orders), rejects every arrival time and binds a value without a unit to the quantity named just before it.

Slots-only narrator. As an alternative to checking numbers, the prompt can forbid them: the model writes no digit or number word and refers to facts by placeholders ({probability}, {rain_mm}, {river_m}, {barrage_m}, {time}, {threshold}, {places}), which the program fills from the state. A reply that still contains a digit, a number word or an unknown placeholder is replaced by the template.

IV. Evaluation Setup

Policy and dispatch path. We evaluated (1) on $p \in \{0, 0.001, \ldots, 1\}$ plus an undefined value, all eight health combinations and both override states ($1{,}002 \times 8 \times 2 =$ 16,032 cases), against five properties (AUTO only if every check passes and $p \ge \tau_a$; undefined p gives UNAVAILABLE; $p < \tau_m$ without override gives MONITOR; $p \ge \tau_a$ with a failed check gives REVIEW; monotone in p when healthy). We then extracted the dispatch decision into a pure function and checked the composed path (policy, hold, dispatch) over every defined probability, health combination, hold and override ($1{,}001 \times 8 \times 2 \times 2 =$ 32,032 cases) for the property that teams are dispatched only when every check passes and the models agree, or an operator overrides. Because p enters both functions only through comparisons with 0.40, 0.50 and 0.60, all of which lie on the grid, the grid contains every equivalence class, so each check is a proof for the extracted functions, not a sample.

Live replies. For 150 seeded states (rainfall 5–250 mm in a monsoon month, both model probabilities from the deployed classifiers, river 200.0–208.5 m, barrage 199.0–206.0 m) we sent the production prompt to gpt-oss-20b and gpt-oss-120b on Groq at temperature 0.85 without fallback (300 replies, all parsed), and stored the verdict of each check. Reference labels follow a written protocol. A number is *supported* if it states a state value for the quantity it describes (honest rounding allowed) or quotes a band threshold; *procedural* numbers (reporting intervals, unit labels, the count of named places, band ranges, deadlines given as orders) are ignored; anything else, including any arrival time, is *unsupported*. The protocol was applied by a script; replies on which it disagreed with any checker were then read, and 4 labels were changed, all to "procedural". Because this reading saw the checkers' verdicts, we also report every result under the script's labels alone.

Prompt designs. For the first 120 of these states and the same models we ran two further prompts on one day (the run was stopped there to limit API use): the production prompt plus one line forbidding arrival times, and the slots-only prompt (233 calls each), plus the unchanged production prompt on the first 50 states (100 calls) to check for drift. These replies were labelled by the script only.

LLM judge. Qwen3 (27B), a family different from the generators, received the structured facts and the turns at temperature 0 and returned whether every number was supported [12]. It saw 320 injected replies (40 states) and the 300 live replies. 23 live calls failed after retries; the deployed system would send these to the template, but we report the judge both with them counted as rejections and without them.

Held-out model family. $\Gamma_{\mathrm{T}}^{(2)}$ was written from the misses on the development replies, frozen and hashed (SHA-256) before replies from ALLaM-2 (7B) were collected for the same 150 states. 132 replies parsed; the other 18 calls, all in the automated band (18 of 123 calls there, none of 27 elsewhere), failed or returned invalid JSON, so the held-out set underrepresents that band. The script labelled the replies; reading changed 8 labels, again all to "procedural".

Claim check. A short pattern check rejects sentences that assert an observed flood ("flood confirmed", "has breached", "water ingress") unless a hedge ("could", "chance of", "expected") precedes them. It was written on the development replies and frozen by hash before any reply of the prompt-design run was read.

Gate. The prototype's rainfall sweep varies 24-h rainfall (0–300 mm) and month for one synthetic gauge with default attributes; 37% of its settings exceed the 99th percentile of 1-day rainfall in the training events (188 mm), so the sweep extrapolates. We therefore also rebuilt the classifiers' split and used its 910 held-out events. A held event goes to an officer; we count the policy model's errors that the gate holds, false dispatches (released, $p \ge \tau_a$, not severe) and missed dispatches (released, $p < \tau_a$, severe), with 2,000-event bootstrap intervals.

TABLE I
CHECKS ON 300 LIVE REPLIES OF GPT-OSS (REFERENCE LABELS)

Check	Recall (95% CI)	FR	Prec.	Calls
Deployed Γ	89.0 (82–94)	5.8	89.8	0
Typed Γ_{T}	93.6 (87–97)	10.5	83.6	0
LLM judge	99.1 (95–100)	14.1	80.0	1
LLM judge*	99.0 (95–100)	6.8	89.3	1

In %. Recall over the 109 unsupported replies; FR: rejections of the 191 supported ones. *23 failed calls excluded (277 replies).

TABLE II
PROMPT DESIGNS, LIVE, FIRST 120 STATES (SCRIPT LABELS)

Prompt	Calls	Unsup.	ETA	Templ.	Claim	Tokens	Words
production prompt*	100	28.0	26.0	26.0	8.0	634	64
+ no arrival times	233	3.4	0.4	2.6	17.6	664	67
slots only	234	1.7	0.4	2.1	5.1	734	57

In % except calls, tokens (prompt + output per call) and words per reply. Templ.: replaced by the template (failed call, or rejected by Γ; for slots, a digit or number word left in the text). *First 50 states only.

V. RESULTS

A. Policy and the Path to Dispatch

The policy satisfied all five properties in every case. The composed check found a fault: the original dispatch step tested the probability band and the hold but never read the action returned by π, so when a health check failed while both models agreed on a high probability, π returned REVIEW and teams were still dispatched; the hold also applied only in simulation mode. The 2,807 violating grid cases are exactly the 401 probabilities at or above τ_a times the 7 health combinations with a failure, without hold or override, i.e. one equivalence class. The count is a grid count, not an exposure estimate: the fault fires whenever a health check fails while $p \geq \tau_a$ and the gate releases. In the prototype's sweep, 119 of 604 settings were in that state, so a data or database outage at any of them would have dispatched; we have no record of how often health checks fail in operation. After the fix (dispatch requires AUTO or OVERRIDE, and the hold applies in both modes) the check found 0 violations, and three regression tests now guard the rule. The finding is an integration error that an end-to-end test would also catch; the point is the method, a property check on the composed path. Behaviour that the pure-function check cannot cover remains: health flags that go stale between check and dispatch, an override that races a tick, and task cancellation. The last one did occur in this code base: a cancelled run loop wrote its final status over the next run, which stopped after about two ticks, and was found only by running the system.

B. Numeric Grounding of Live Replies

109 of the 300 replies (36.3%; 33.3% for gpt-oss-20b and 39.3% for gpt-oss-120b) stated at least one unsupported number, and 106 of these 109 (97%) were invented arrival times such as "ETA 10 minutes"; the prompt contains no ETA. Probabilities, rainfall, water levels and times were almost always quoted correctly. Table I compares the checks.

The deployed guard missed 12 unsupported replies, mostly arrival times in words ("ETA about ten minutes") or equal to the current clock time. The typed guard caught more but rejected 20 supported replies, mostly for procedural numbers or values without a unit ("barrage 202.26"); in paired tests it added false rejections significantly (McNemar $p = 0.004$) but not recall ($p = 0.062$). Counting its 23 failed calls as rejections, the LLM judge rejected 14.1% of supported replies; 15 of those calls were on supported replies. On the 277 replies it answered, it missed 1 unsupported reply (recall 99.0%) and rejected 6.8% of supported ones, fewer than the typed guard (10.2%) on the same replies. The judge recognises an invented ETA in any wording, but it costs one model call per message, failed on 23 of 300 calls, and is itself a model whose verdicts cannot be checked exhaustively.

Removing the $\times 100$ and $/100$ copies from $\tilde{F}$ left recall unchanged on the development replies (89.0%) and raised false rejections from 5.8% to 6.3%; on the held-out model, recall rose from 96.1% to 97.4% and false rejections from 7.1% to 12.5%. Under the script's labels alone (113 unsupported development replies instead of 109), the guards' recall was 88.5%, 93.8% and 96.5%, close to Table I.

C. Prompt Designs

Table II gives the two alternative prompts. Re-run on the same day, the production prompt still produced an unsupported number in 28.0% of replies (the script's label for the original run: 37.7%). One added line forbidding arrival times lowered this to 3.4% and arrival times from 26.0% to 0.4% of replies, at a cost of 30 prompt and output tokens per call. The slots-only prompt left a digit or number word in 2.1% of replies, which therefore fall back to the template (2.1% of calls in all). Of the replies that passed this check, one still contained an unsupported number after rendering. Its replies were 57 words long against 64 and used 734 tokens per call against 634; whether the placeholder style reads as well on a radio net is a question for officers.

D. Injected Errors

Table III compares the checks on injected errors: one grounded reply and seven copies with one wrong value per state, passed through the production code with the HTTP client stubbed. The deployed guard rejected 96.0% of wrong digit strings with no false rejections and none of the wrong values written in words; the typed guard rejected 98.8% and 81.5% of number words. The 48 deployed misses were collisions: 39 with a genuine fact for another quantity (with a probability of 87.3%, an invented "ETA 87 minutes" passed), 8 with a band threshold and 1 with a $\times 100$ copy. The judge was the weakest check on small probability errors, which change a single digit.

E. A Held-Out Model Family

Table IV separates the development replies, on which $\Gamma_{\mathrm{T}}^{(2)}$ was built and is perfect by construction, from the frozen test on ALLaM-2. 76 of its 132 replies (57.6%) contained

TABLE III
DETECTION RATE (%) BY INJECTED ERROR TYPE

Injected error	200 states ($n = 200$ per row) Γ	facts only	Γ_{T}	40 states ($n = 40$) Γ	Γ_{T}	judge
Grounded (false rej.)	0.0	0.0	0.0	0.0	0.0	0.0
Probability ±1–5	94.0	95.0	97.0	97.5	100.0	85.0
Probability ±10–30	95.5	96.5	97.0	92.5	95.0	97.5
Rainfall 0.5–1.5×	97.0	97.5	100.0	95.0	100.0	100.0
River ±0.5–2 m	98.0	98.0	98.5	100.0	100.0	100.0
Invented head-count	97.5	98.0	100.0	100.0	100.0	100.0
Invented ETA	94.0	95.0	100.0	95.0	100.0	100.0
Wrong value in words	0.0	0.0	81.5	0.0	92.5	90.0
All numeric errors	96.0	96.7	98.8	96.7	99.2	97.1

facts only: Γ without band-rule numbers. Grounded row: false rejections. The 95% Wilson intervals of the judge column are at most ±14 points.

TABLE IV
GUARDS ON DEVELOPMENT REPLIES AND ON A HELD-OUT MODEL FAMILY

Check	gpt-oss, 300 (dev.) Recall	FR	ALLaM-2, 132 (held out) Recall (95% CI)	FR
Deployed Γ	89.0	5.8	96.1 (89–99)	7.1
Typed Γ_{T}	93.6	10.5	100.0 (95–100)	17.9
Typed v2 (frozen)	100.0	0.0	98.7 (93–100)	12.5

In %. $\Gamma_{\mathrm{T}}^{(2)}$ was frozen by hash before any ALLaM-2 reply was read.

an unsupported number, and most were not arrival times but invented quantities: head-counts, rates of rise, durations, coordinates and casualty estimates (39% of all its replies, 67% of its unsupported ones). The shift is sharp: arrival times appeared in 32–39% of gpt-oss replies but 4.5% of ALLaM-2 replies, which also misquoted a probability in 16%, an error the development models never made.

On ALLaM-2 the frozen guard caught 98.7% against 96.1% for the deployed guard, a difference of two replies: it alone caught 3 and the deployed guard alone 1 (McNemar $p = 0.62$). It also rejected 7 correct replies against 4 (12.5% against 7.1%; $p = 0.25$), all of them procedural labels such as "Team 1" or "level 2 alert" that the development models never wrote. On this held-out set, therefore, neither guard is shown to be better. Under the script's labels alone, which count those labels as unsupported, neither guard falsely rejected any held-out reply, which shows how much the false-rejection rates depend on the procedural judgement. For example, "Team 1 (Rescue) will head to" is correct but was rejected, while "begun flooding with 134 affected" invents a head-count. Deployed, a guard's rejections become template fallbacks: 38.7% of gpt-oss-20b calls, 33.3% of gpt-oss-120b and 63.3% of ALLaM-2 calls ended in the template. Checking one reply with $\Gamma_{\mathrm{T}}^{(2)}$ took 371 μs on a laptop.

F. Qualitative Over-Claims

43 development replies (14.3%), all in the automated band, asserted that a flood "is confirmed" or that water "has breached", although the state holds only a probability; such a claim can trigger or cancel an evacuation as readily as a wrong number. No numeric check is designed to see them: the deployed guard passed 27 of the 43 (63%), and the others rejected some only because the same reply also contained a bad number (judge: 20 passed). The wording comes partly from our own prompt, whose automated-band rule says "flood occurred", and the deployed template says "Flood has occurred"; the fallback itself over-claims. The frozen claim check flagged 8.0%, 17.6% and 5.1% of the replies to the three prompts of Table II, which it had never seen. Our grounding claim is therefore limited to numbers; over-claims need a rule on assertion words and, more simply, a prompt and template that speak of probability.

TABLE V
GATE RULES ON 910 HELD-OUT EVENTS (POLICY MODEL: XGBOOST)

Rule	Held %	Errors caught % (95% CI)	False disp.	Missed disp.	Cost / 100 1:1	1:5
no gate	0.0	0	93	133	24.8	83.3
$\delta = 0.15$ (deployed)	26.9	40.3 (34–47)	38	97	20.2	62.9
δ_{95} (train CV)	9.1	17.7 (13–23)	60	126	22.3	77.6
straddle only	7.6	15.9 (11–21)	62	128	22.4	78.7
calibrated, t^* (no gate)	0.0	0	320	27	38.1	50.0

Of 226 policy-model errors. Cost: false dispatch 1, missed dispatch 1 or 5, review 0.2 (assumed; held events assumed decided correctly).

G. Dual-Model Gate

On the prototype's sweep (604 settings) the gate held 69.7% of settings, including 73.7% of those where both models exceeded τ_a, so automatic dispatch was almost unreachable there. On the 910 held-out events (Table V) the XGBoost model was over-confident (ECE 0.108, against 0.044 for the Random Forest; above τ_a it predicted 0.84 where severe events occurred at 0.67), so the two models differ systematically. The deployed gate held 26.9% of events and caught 91 of the policy model's 226 errors (40.3%). The straddle rule held fewer events (69 against 245) with a higher error rate among them (52.2% against 37.1%), but that is precision: it caught only 36 errors (15.9%), so replacing the gap term with it would let 55 more errors reach action (95% interval 42–69). A tolerance learned on training data alone, $\delta_{95} = 0.28$ (the sweep stops holding cases on which both models agree at $\delta = 0.31$), behaved like the straddle rule. Under assumed costs, with a review costing 0.2 of a false dispatch and the officer assumed always right, the deployed gate had the lowest cost when a missed dispatch costs as much as a false one (20.2 against 24.8 per 100 events without a gate). When a missed dispatch costs five times as much, a single isotonic-calibrated model without any gate, its threshold ($t^* = 0.18$) set on training folds, was cheapest (50.0 against 62.9). Disagreement thus carries information, but whether a gate pays depends on costs that only an operations room can set.

VI. DISCUSSION AND LIMITATIONS

The results locate the weak points in the glue between components. A dispatch step bypassed a verified policy, a gate tolerance was never related to the calibration of the

models it compares, an untyped guard treated every number as interchangeable, and the template meant to absorb failures carried its own over-claim. None of these showed up when components were tested one at a time. Which errors appear depends on the model: the development models added arrival times, the held-out model added head-counts and misquoted facts. A check designed around one model's habits has to be tested again on the next one, and a fail-closed fallback hides what it absorbs: while we prepared this study, retired API models and a token budget too small for reasoning models silently sent every reply to the template, which only the fallback rate revealed.

The limits are clear. All quantitative rates come from one prompt family, 150 states, temperature 0.85 and three open-weight models from two families; no frontier or other common open model was tested, and the rates will change with all of these. Reference labels were assigned by one annotator, who saw the checkers' verdicts while reading disagreements; results under the script's labels are reported, and an independent second annotation is the main open item. River and barrage values were sampled from operating ranges, not recorded events. The policy and dispatch checks prove properties of extracted functions, not of the asynchronous pipeline. The classifiers predict severity from a random split of events, and the gate costs are assumptions. No officer has used the system in an exercise.

For a deployment we draw four rules. Prefer a narrator that does not write numbers, or failing that forbid the quantities the state lacks. Speak of probabilities in prompts and templates, and check assertion words as well as numbers. Measure the template-fallback rate per model, the only sign of a guard that has started to reject correct traffic. And re-run a labelled held-out test whenever the model behind the API changes.

VII. Conclusion

In a flood-response pipeline where deterministic rules decide and an LLM narrates, a composed check found that dispatch bypassed the verified policy, a fault a component test could not reveal. In 300 live replies of two open-weight LLMs, 36.3% stated an unsupported number, 97% of them arrival times; the deployed guard caught 89.0% and a typed guard 93.6%, at the cost of more false rejections. Forbidding arrival times in the prompt cut unsupported replies from 28.0% to 3.4%, and a narrator that writes only placeholders left 1 unsupported number in 234 replies, at the price of 2.1% template fallbacks. A revised guard frozen before a third model was queried was not significantly better than the deployed one on that model, and qualitative over-claims, partly seeded by our own prompt and template, escape every numeric check. The disagreement gate caught 40.3% of the classifier's errors, and whether it is worth its holds depends on costs set by the operations room. LLM narration can serve an operations net only if it stays outside the decision path and is checked against the state it describes, in its numbers and in its claims.

Acknowledgment

The authors thank the INDOFLOODS dataset authors and Open-Meteo for openly available data. In line with the IEEE policy on AI-generated content, the authors disclose that Claude (Anthropic) was used to help edit and restructure the text of all sections and to draft the experiment and plotting scripts. All experiments use the authors' own software and can be reproduced with fixed seeds; the live LLM replies are stored with the scripts. The authors checked every number, claim and reference and take full responsibility for the content.

References

[1] Z. Ji *et al.*, "Survey of hallucination in natural language generation," *ACM Comput. Surv.*, vol. 55, no. 12, Art. no. 248, 2023, doi: 10.1145/3571730.

[2] S. Yao *et al.*, "ReAct: Synergizing reasoning and acting in language models," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2023.

[3] Q. Wu *et al.*, "AutoGen: Enabling next-gen LLM applications via multi-agent conversation," in *Proc. Conf. Lang. Model. (COLM)*, 2024.

[4] R. Parasuraman, T. B. Sheridan, and C. D. Wickens, "A model for types and levels of human interaction with automation," *IEEE Trans. Syst., Man, Cybern. A, Syst. Humans*, vol. 30, no. 3, pp. 286–297, 2000, doi: 10.1109/3468.844354.

[5] N. G. Leveson, *Engineering a Safer World: Systems Thinking Applied to Safety*. Cambridge, MA, USA: MIT Press, 2011.

[6] OASIS, "Common Alerting Protocol version 1.2," OASIS Standard, Jul. 2010.

[7] L. Wang *et al.*, "A survey on large language model based autonomous agents," *Front. Comput. Sci.*, vol. 18, no. 6, Art. no. 186345, 2024, doi: 10.1007/s11704-024-40231-1.

[8] P. Lewis *et al.*, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Adv. Neural Inf. Process. Syst. 33*, 2020, pp. 9459–9474.

[9] S. Min *et al.*, "FActScore: Fine-grained atomic evaluation of factual precision in long form text generation," in *Proc. Conf. Empir. Methods Nat. Lang. Process. (EMNLP)*, 2023, pp. 12076–12100, doi: 10.18653/v1/2023.emnlp-main.741.

[10] P. Manakul, A. Liusie, and M. J. F. Gales, "SelfCheckGPT: Zero-resource black-box hallucination detection for generative large language models," in *Proc. Conf. Empir. Methods Nat. Lang. Process. (EMNLP)*, 2023, pp. 9004–9017, doi: 10.18653/v1/2023.emnlp-main.557.

[11] S. Dhuliawala *et al.*, "Chain-of-verification reduces hallucination in large language models," in *Findings Assoc. Comput. Linguist. (ACL)*, 2024, pp. 3563–3578, doi: 10.18653/v1/2024.findings-acl.212.

[12] L. Zheng *et al.*, "Judging LLM-as-a-judge with MT-Bench and Chatbot Arena," in *Adv. Neural Inf. Process. Syst. 36 (Datasets Benchmarks Track)*, 2023.

[13] T. Rebedea, R. Dinu, M. N. Sreedhar, C. Parisien, and J. Cohen, "NeMo Guardrails: A toolkit for controllable and safe LLM applications with programmable rails," in *Proc. Conf. Empir. Methods Nat. Lang. Process.: Syst. Demonstrations*, 2023, pp. 431–445, doi: 10.18653/v1/2023.emnlp-demo.40.

[14] Y. Zhang, S. Y. Emma, A. L. J. En, and J. S. Dong, "RvLLM: LLM runtime verification with domain knowledge," in *Adv. Neural Inf. Process. Syst. 38*, 2025, pp. 89374–89394, doi: 10.52202/085713-2693.

[15] H. Wang, C. M. Poskitt, and J. Sun, "AgentSpec: Customizable runtime enforcement for safe and reliable LLM agents," in *Proc. IEEE/ACM 48th Int. Conf. Softw. Eng. (ICSE)*, 2026, pp. 2938–2950, doi: 10.1145/3744916.3764546.

[16] M. Alshiekh, R. Bloem, R. Ehlers, B. Könighofer, S. Niekum, and U. Topcu, "Safe reinforcement learning via shielding," in *Proc. AAAI Conf. Artif. Intell.*, vol. 32, no. 1, 2018, pp. 2669–2678, doi: 10.1609/aaai.v32i1.11797.

[17] B. Lakshminarayanan, A. Pritzel, and C. Blundell, "Simple and scalable predictive uncertainty estimation using deep ensembles," in *Adv. Neural Inf. Process. Syst. 30*, 2017, pp. 6402–6413.

[18] L. Breiman, "Random forests," *Mach. Learn.*, vol. 45, no. 1, pp. 5–32, 2001, doi: 10.1023/A:1010933404324.

[19] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowl. Discov. Data Min.*, 2016, pp. 785–794, doi: 10.1145/2939672.2939785.